

Курс Stepik. Этап 3

Прохождение обучающего курса на платформе Stepik

Лебедев Сергей Алексеевич

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение	8
3.1	Раздел 3.1. Текстовый редактор vim	8
3.1.1	Шаг 1. Выход из редактора vim	8
3.1.2	Шаг 2. Разница между word и WORD в vim	9
3.1.3	Шаг 3. Дополнение к заданию о word и WORD	9
3.1.4	Шаг 4. Преобразование строки в vim	10
3.1.5	Шаг 5. Продолжение задания о преобразовании строки . . .	11
3.1.6	Шаг 6. Замена слова в каждой строке через :%s	11
3.1.7	Шаг 7. Режим выделения Visual в vim	12
3.1.8	Шаг 8. Продолжение вопроса о режиме Visual	13
3.2	Раздел 3.2. Скрипты на bash: основы	13
3.2.1	Шаг 9. История команд при вложенных оболочках	13
3.2.2	Шаг 10. Абсолютный путь до файла после работы скрипта .	14
3.2.3	Шаг 11. Допустимые имена переменных в bash	15
3.2.4	Шаг 12. Программа для вывода позиционных аргументов . .	15
3.3	Раздел 3.3. Скрипты на bash: ветвления и циклы	16
3.3.1	Шаг 13. Синтаксис условий в bash	16
3.3.2	Шаг 14. Вывод скрипта при разных значениях переменной .	17
3.3.3	Шаг 15. Скрипт с конструкцией case для вывода числа студентов	17
3.3.4	Шаг 16. Количество выводов start и finish в цикле	18
3.3.5	Шаг 17. Скрипт для определения возрастной группы	19
3.4	Раздел 3.4. Скрипты на bash: разное	19
3.4.1	Шаг 18. Команда let в bash	19
3.4.2	Шаг 19. Вывод команды echo с подстановкой команды	20
3.4.3	Шаг 20. Проверка кода возврата программы	21
3.4.4	Шаг 21. Вывод скрипта после десяти вызовов функции . . .	21
3.4.5	Шаг 22. Функция НОД по алгоритму Евклида	22
3.4.6	Шаг 23. Калькулятор для двух чисел	23
3.5	Раздел 3.5. Продвинутый поиск и редактирование	25
3.5.1	Шаг 24. Опция -iname в команде find	25
3.5.2	Шаг 25. Опции -path и -name в команде find	26
3.5.3	Шаг 26. Опции -mindepth и -maxdepth в find	26
3.5.4	Шаг 27. Опции -A, -B, -C в команде grep	27

3.5.5	Шаг 28. Регулярное выражение в grep -E	28
3.5.6	Шаг 29. Поведение sed без опции -n	28
3.5.7	Шаг 30. Замена аббревиатур через sed	29
3.5.8	Шаг 31. Продолжение задания по sed	30
3.6	Раздел 3.6. Строим графики в gnuplot	30
3.6.1	Шаг 32. Опция gnuplot для сохранения графиков	30
3.6.2	Шаг 33. Команда set key autotitle columnhead	31
3.6.3	Шаг 34. Конкатенация строк для подписей на оси OX	32
3.6.4	Шаг 35. Модификация скрипта move.rot в gnuplot	32
4	Выводы	34
	Список литературы	35

Список иллюстраций

3.1	Вопрос о выходе из vim	8
3.2	Вопрос о разнице word и WORD	9
3.3	Продолжение вопроса о word и WORD	10
3.4	Вопрос о преобразовании строки в vim	10
3.5	Продолжение вопроса о преобразовании строки	11
3.6	Вопрос о замене слова в каждой строке	12
3.7	Вопрос о режиме Visual в vim	12
3.8	Продолжение вопроса о режиме Visual	13
3.9	Вопрос об истории команд в вложенных оболочках	14
3.10	Вопрос об абсолютном пути после скрипта	14
3.11	Вопрос о допустимых именах переменных	15
3.12	Задание на вывод позиционных аргументов	16
3.13	Вопрос о синтаксисе условий в bash	16
3.14	Вопрос о выводе скрипта при разных значениях	17
3.15	Задание на скрипт с конструкцией case	18
3.16	Вопрос о количестве выводов в цикле	18
3.17	Задание на скрипт определения возрастной группы	19
3.18	Вопрос о команде let в bash	20
3.19	Вопрос о выводе echo с подстановкой команды	20
3.20	Вопрос о проверке кода возврата	21
3.21	Вопрос о выводе после вызовов функции	22
3.22	Задание на функцию НОД	23
3.23	Задание на калькулятор	25
3.24	Вопрос об опции -iname в find	25
3.25	Вопрос об опциях -path и -name в find	26
3.26	Вопрос об опциях -mindepth и -maxdepth	27
3.27	Вопрос об опциях -A, -B, -C в grep	27
3.28	Вопрос о регулярном выражении в grep -E	28
3.29	Вопрос о поведении sed без опции -n	29
3.30	Задание на замену аббревиатур через sed	29
3.31	Продолжение задания по sed	30
3.32	Вопрос об опции -persist в gnuplot	31
3.33	Вопрос о команде set key autotitle columnhead	31
3.34	Задание на конкатенацию строк для подписей осей	32
3.35	Задание на модификацию скрипта move.rot	33

Список таблиц

1 Цель работы

Пройти третью часть обучающего курса на платформе Stepik, посвящённой продвинутым темам работы в Linux: текстовому редактору vim, написанию скриптов на bash, продвинутому поиску и редактированию текста, а также построению графиков в gnuplot.

2 Задание

Выполнить все задания третьей части курса на платформе Stepik по разделам: 3.1 - текстовый редактор vim, 3.2 - скрипты на bash (основы), 3.3 - ветвления и циклы, 3.4 - разное, 3.5 - продвинутый поиск и редактирование, 3.6 - построение графиков в gnuplot.

3 Выполнение

3.1 Раздел 3.1. Текстовый редактор vim

3.1.1 Шаг 1. Выход из редактора vim

Вопрос о том, какие клавиши нужно нажать, чтобы выйти из vim сразу после открытия файла. Выбран верный ответ: **: , затем q, затем Enter**. При открытии файла vim находится в нормальном режиме, поэтому вводить i для перехода в режим вставки не нужно (рис. 3.1).

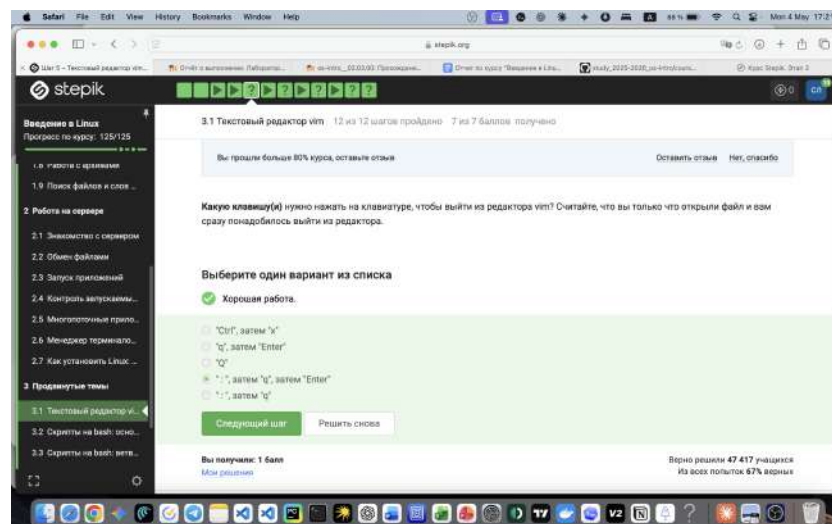


Рис. 3.1: Вопрос о выходе из vim

3.1.2 Шаг 2. Разница между word и WORD в vim

Вопрос о верных утверждениях про перемещение по word (команда w) и WORD (команда W). Верные ответы: в строке 5 «больших слов» (WORD); чтобы попасть в конец строки, нужно меньше нажатий на W, чем на w; нажимая только W, нельзя переместить курсор на . (рис. 3.2).

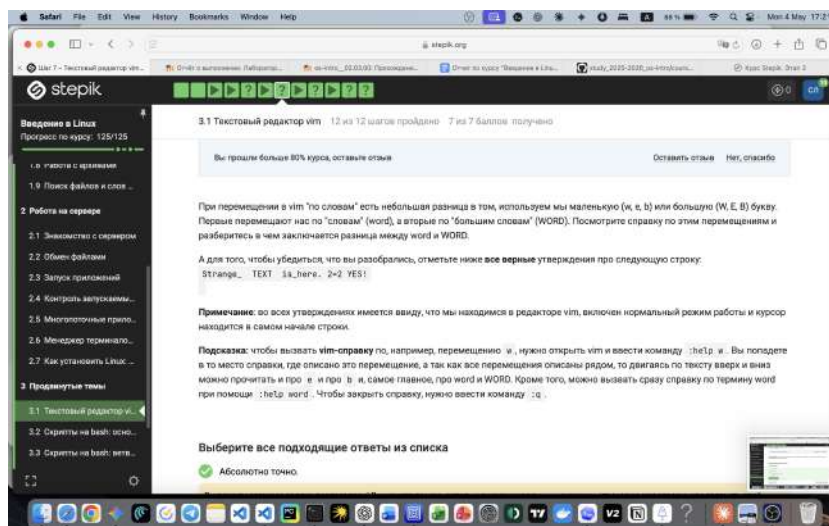


Рис. 3.2: Вопрос о разнице word и WORD

3.1.3 Шаг 3. Дополнение к заданию о word и WORD

Продолжение задания о перемещении по словам в `vim`. Подтверждены те же верные утверждения о разнице команд `w` и `W` (рис. 3.3).

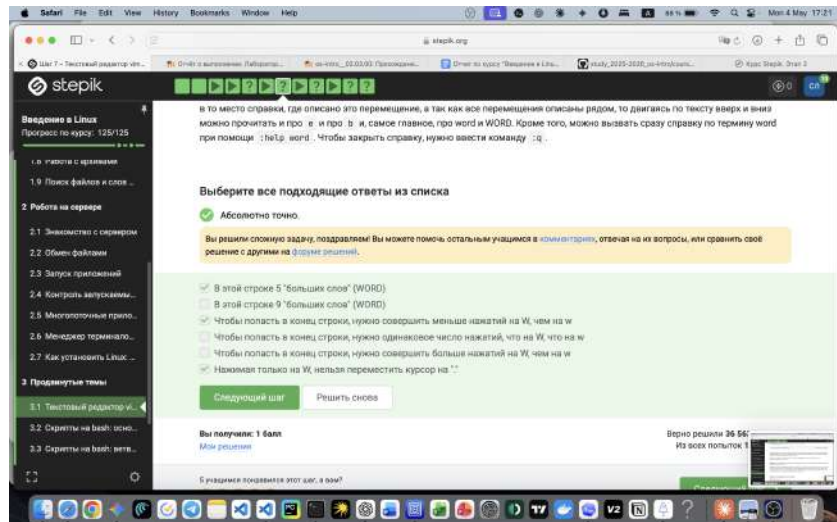


Рис. 3.3: Продолжение вопроса о word и WORD

3.1.4 Шаг 4. Преобразование строки в vim

Задание: преобразовать строку one two three four five в three four four four five. Верные варианты команд: d2wwywPp, ddithree four four four five Esc, d2w bifour four Esc (рис. 3.4).

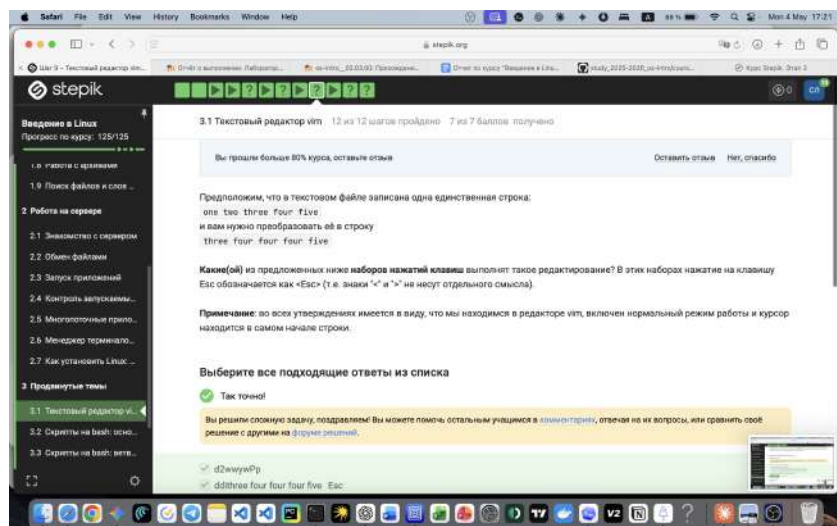


Рис. 3.4: Вопрос о преобразовании строки в vim

3.1.5 Шаг 5. Продолжение задания о преобразовании строки

Дополнительные верные варианты команд для преобразования строки one two three four five в three four four four five (рис. 3.5).

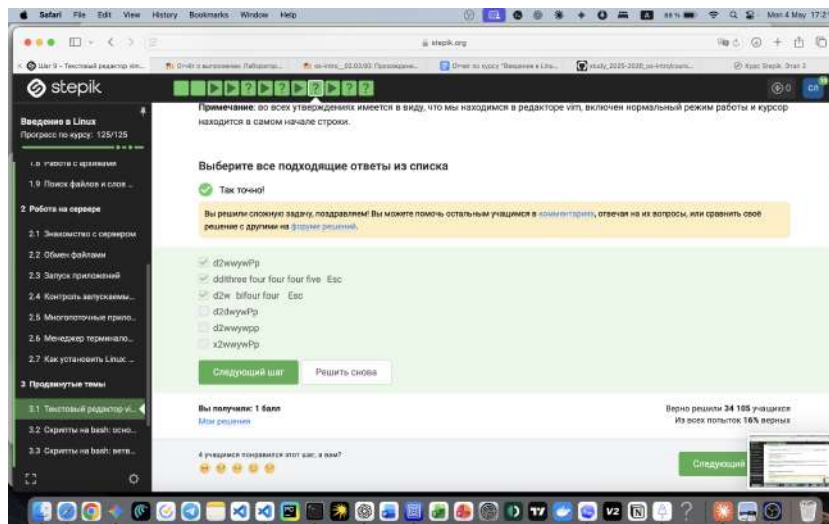


Рис. 3.5: Продолжение вопроса о преобразовании строки

3.1.6 Шаг 6. Замена слова в каждой строке через :%s

Задание: найти команду для замены первого слова Windows на Linux в каждой строке файла. Введён верный ответ:

```
:%s/Windows/Linux/
```

Команда без флага g заменяет только первое вхождение в каждой строке (рис. 3.6).

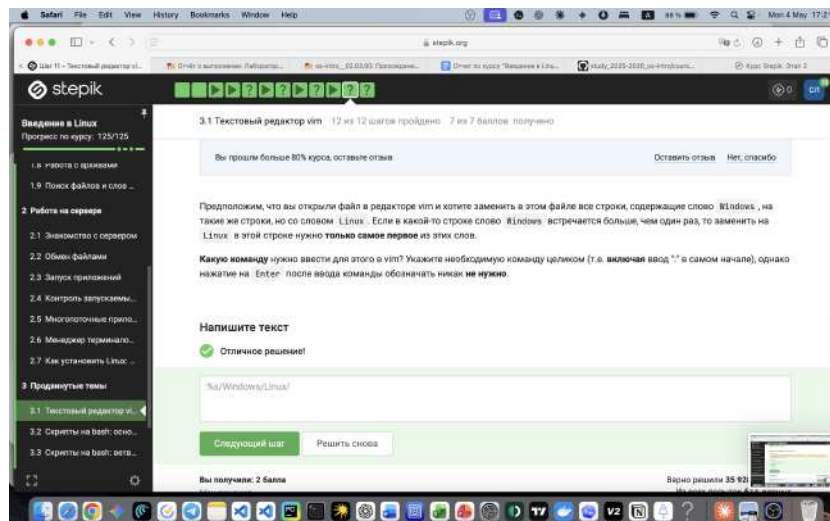


Рис. 3.6: Вопрос о замене слова в каждой строке

3.1.7 Шаг 7. Режим выделения Visual в vim

Вопрос о верных утверждениях про режим выделения (Visual). Верные ответы: внизу редактора отображается надпись -- VISUAL --; режим открывается из нормального режима по нажатию v; доступны команды d (удалить) и y (скопировать); можно использовать команды перемещения (W, e, \$ и др.) (рис. 3.7).

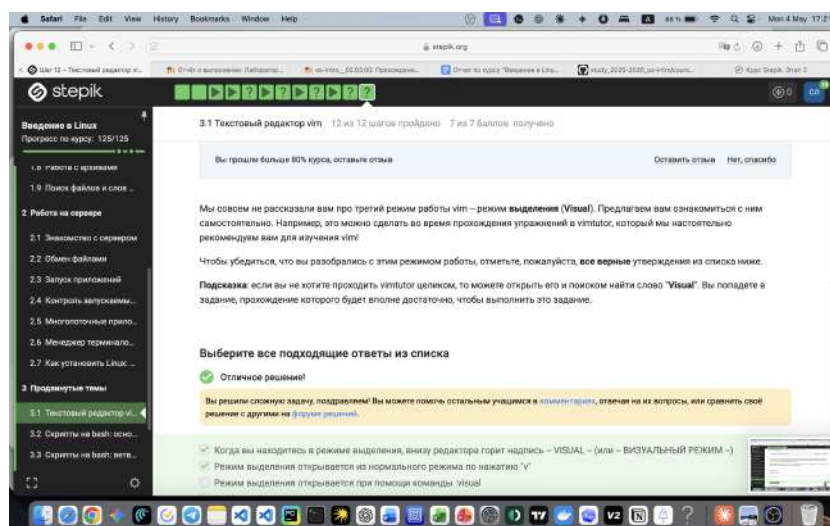


Рис. 3.7: Вопрос о режиме Visual в vim

3.1.8 Шаг 8. Продолжение вопроса о режиме Visual

Дополнение к заданию о режиме Visual - подтверждены те же верные утверждения (рис. 3.8).

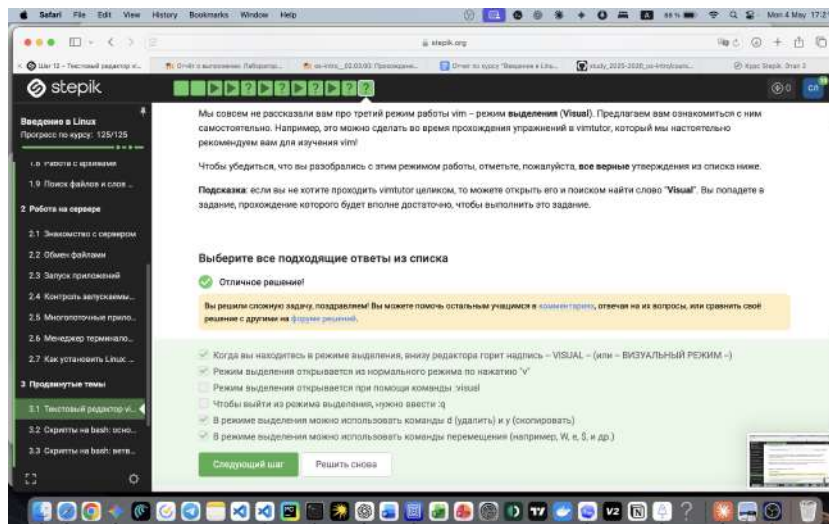


Рис. 3.8: Продолжение вопроса о режиме Visual

3.2 Раздел 3.2. Скрипты на bash: основы

3.2.1 Шаг 9. История команд при вложенных оболочках

Вопрос о том, из какой оболочки будут появляться команды в истории при запуске цепочки `bash -> sh -> bash`. Выбран верный ответ: **только из набора C** - каждая оболочка ведёт собственную историю команд (рис. 3.9).

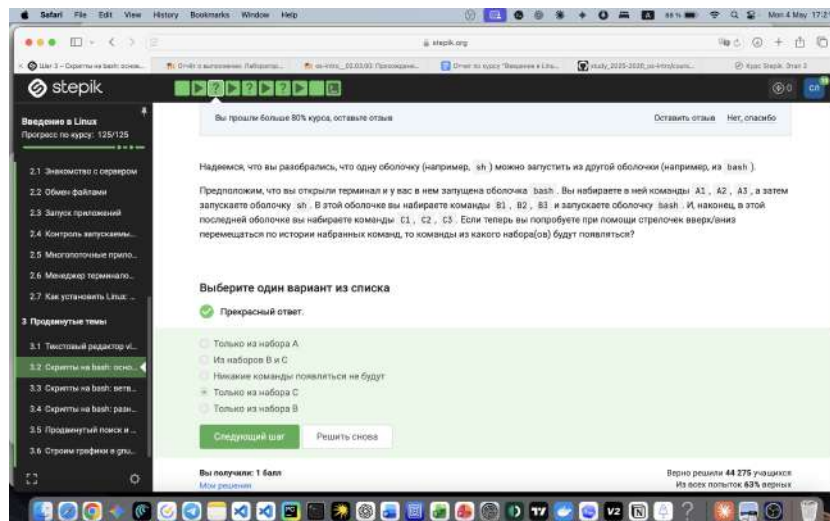


Рис. 3.9: Вопрос об истории команд в вложенных оболочках

3.2.2 Шаг 10. Абсолютный путь до файла после работы скрипта

Задание: определить абсолютный путь до файла `file1.txt` после выполнения скрипта. Введён верный ответ: `/home/bi/file1.txt` (рис. 3.10).

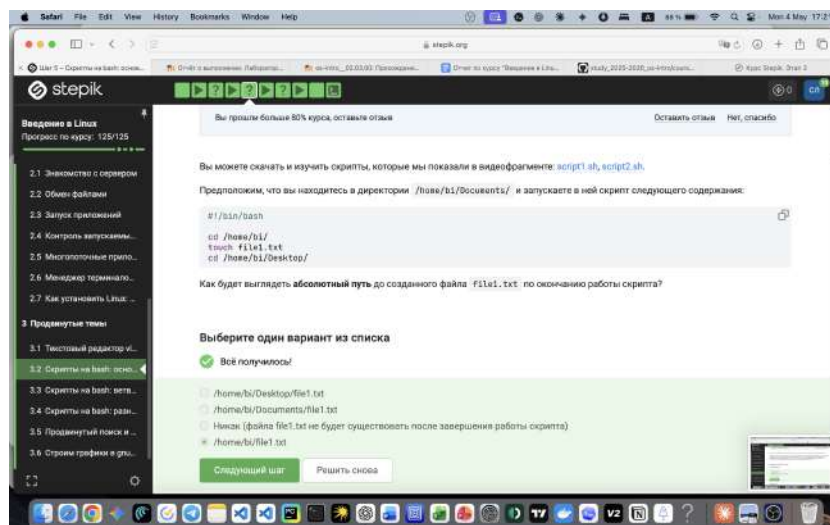


Рис. 3.10: Вопрос об абсолютном пути после скрипта

3.2.3 Шаг 11. Допустимые имена переменных в bash

Вопрос о допустимых именах переменных в bash. Верные варианты: VARIable, variable_123, __variable, _variable, variable. Имена не могут начинаться с цифры и не могут содержать дефис или пробел (рис. 3.11).

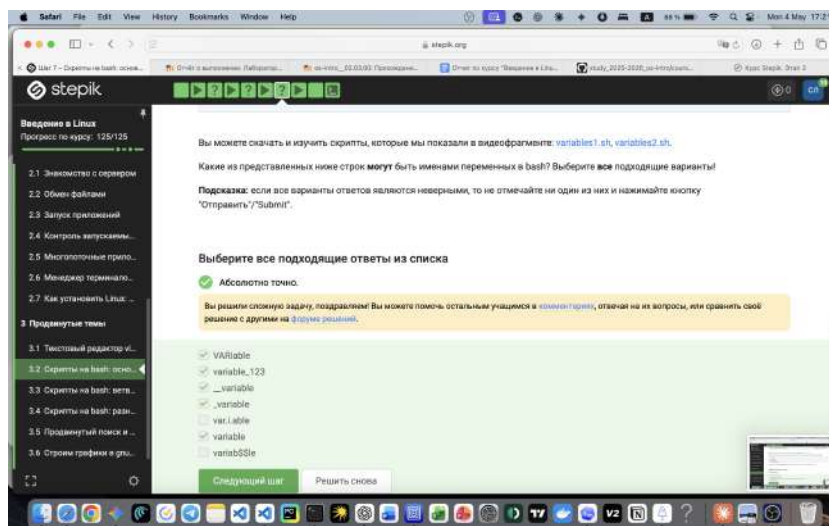


Рис. 3.11: Вопрос о допустимых именах переменных

3.2.4 Шаг 12. Программа для вывода позиционных аргументов

Практическое задание: написать программу, которая выводит позиционные аргументы в формате \$1=значение \$2=значение. Введён верный ответ:

```
echo "Arguments are: \$1=$1 \$2=$2"
```

Получено подтверждение правильности (рис. 3.12).

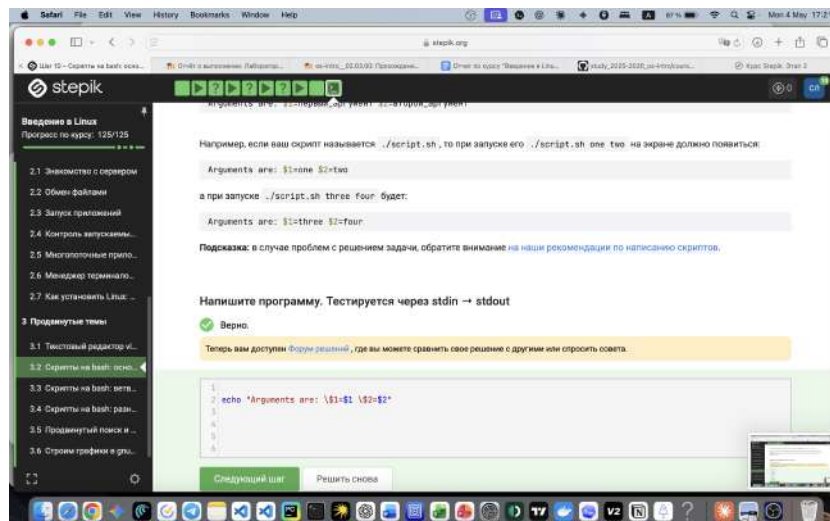


Рис. 3.12: Задание на вывод позиционных аргументов

3.3 Раздел 3.3. Скрипты на bash: ветвления и циклы

3.3.1 Шаг 13. Синтаксис условий в bash

Вопрос о подходящих вариантах синтаксиса условий []. Верные варианты: `$# -ge 0`, `-n $0`, `5 -ge 5`, `-s $0`, `! (4 -le 3)`, `$var1 == $var2` || `$var1 != $var2` (рис. 3.13).



Рис. 3.13: Вопрос о синтаксисе условий в bash

3.3.2 Шаг 14. Вывод скрипта при разных значениях переменной

Задание: определить, что выведет скрипт при запуске с `var=3`, а затем с `var=5`.
Верный ответ: **сначала four, потом four** - в обоих случаях условие приводит к одному результату (рис. 3.14).

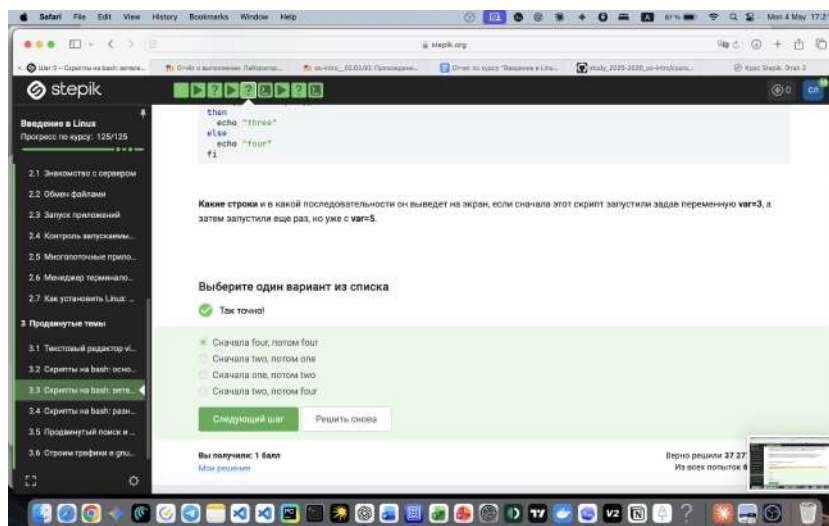


Рис. 3.14: Вопрос о выводе скрипта при разных значениях

3.3.3 Шаг 15. Скрипт с конструкцией case для вывода числа студентов

Практическое задание: написать скрипт с конструкцией `case` для вывода количества студентов в зависимости от переданного аргумента. Скрипт содержит варианты от 0) до 4) и вариант *) `echo "A lot of students"` ;; для всех остальных значений (рис. 3.15).

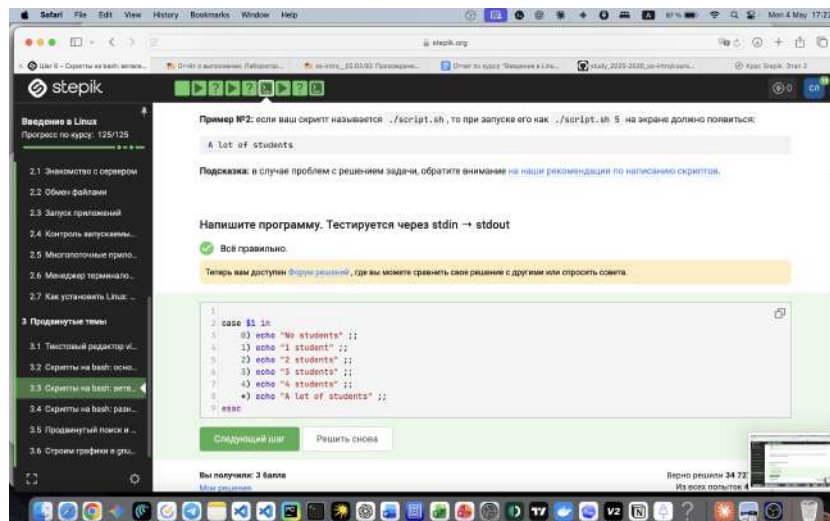


Рис. 3.15: Задание на скрипт с конструкцией case

3.3.4 Шаг 16. Количество выводов start и finish в цикле

Вопрос о том, сколько раз выведется `start` и `finish` в цикле с условием `continue`.
Верный ответ: **5 раз start и 4 раза finish** - команда `continue` пропускает вывод `finish` на одной итерации (рис. 3.16).

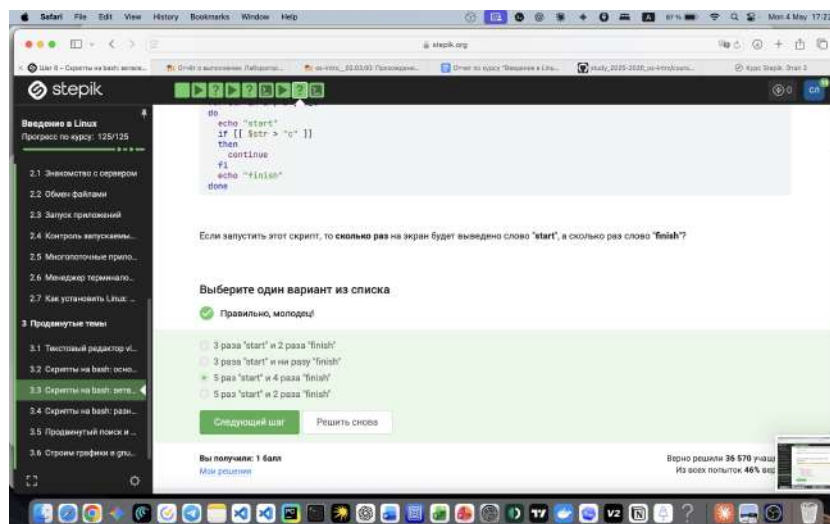


Рис. 3.16: Вопрос о количестве выводов в цикле

3.3.5 Шаг 17. Скрипт для определения возрастной группы

Практическое задание: написать скрипт для определения возрастной группы пользователя (child, youth, adult). Скрипт реализован с бесконечным циклом `while true`, проверками `if [[ -z ... ]]` и `elif` для разбора введенного возраста (рис. 3.17).

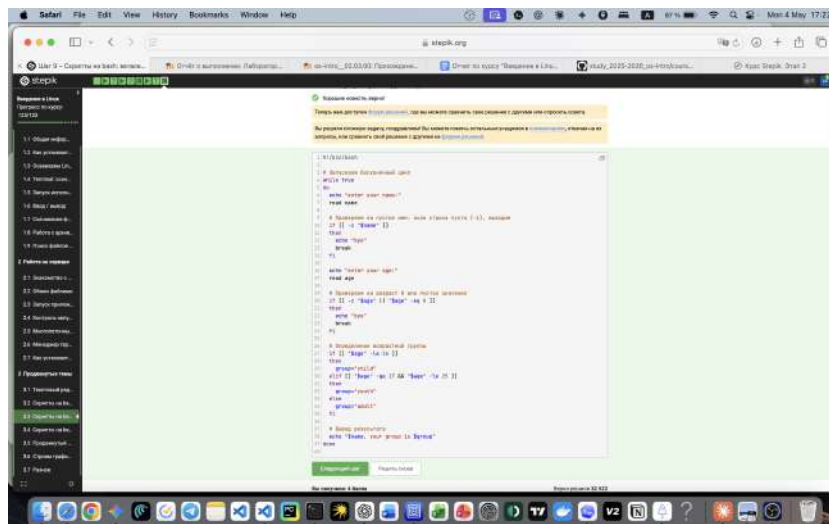


Рис. 3.17: Задание на скрипт определения возрастной группы

3.4 Раздел 3.4. Скрипты на bash: разное

3.4.1 Шаг 18. Команда `let` в `bash`

Вопрос о допустимых вариантах команды `let`. Верные варианты: `let a=$a+$b`, `let "a=$a+$b"`, `let a=a+b`. В `let` можно обращаться к переменным как с `$`, так и без него (рис. 3.18).

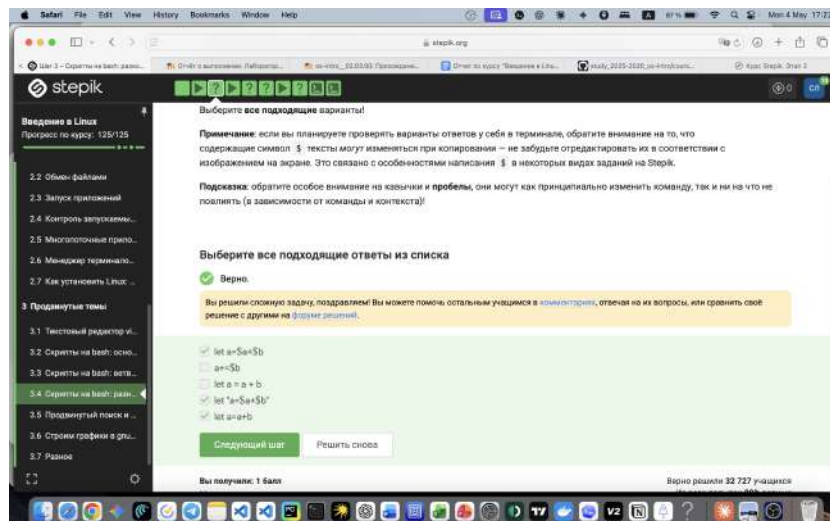


Рис. 3.18: Вопрос о команде let в bash

3.4.2 Шаг 19. Вывод команды echo с подстановкой команды

Задание: определить, что выведет команда `echo "\pwd"`, если до этого выполнен `cd /home/bi/`. Верный ответ: `**/home/bi/**` - обратные кавычки выполняют команду `pwd` и подставляют её вывод (рис. 3.19).

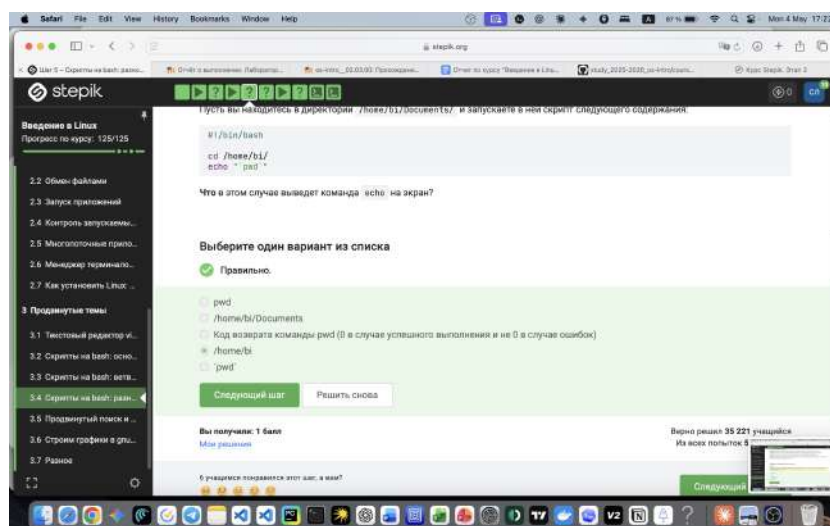


Рис. 3.19: Вопрос о выводе echo с подстановкой команды

3.4.3 Шаг 20. Проверка кода возврата программы

Вопрос о способах запустить программу и выполнить действия, если её код возврата равен 0. Верные варианты: сначала запустить `program`, затем `if [[ $? -eq 0 ]]`; а также конструкция `if \program > some_file.txt` (рис. 3.20).

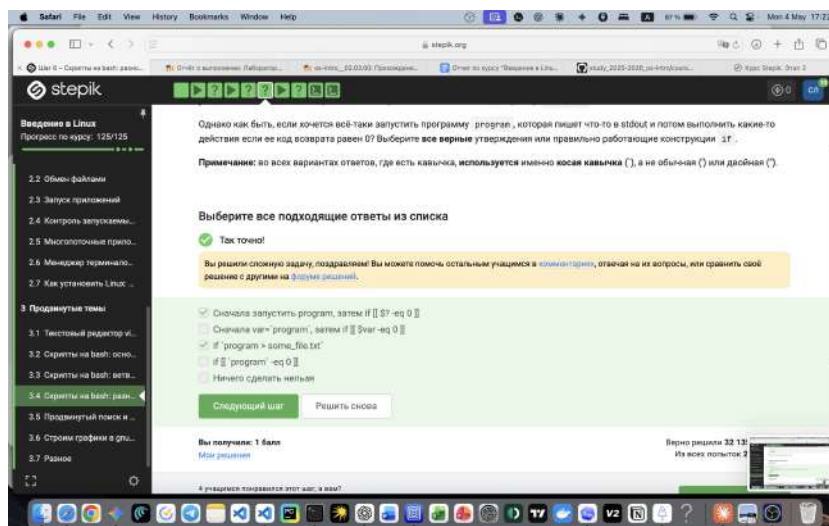


Рис. 3.20: Вопрос о проверке кода возврата

3.4.4 Шаг 21. Вывод скрипта после десяти вызовов функции

Задание: определить, что выведет скрипт на экран после десяти вызовов функции `counter`. Верный ответ: **counters are and 110** (рис. 3.21).

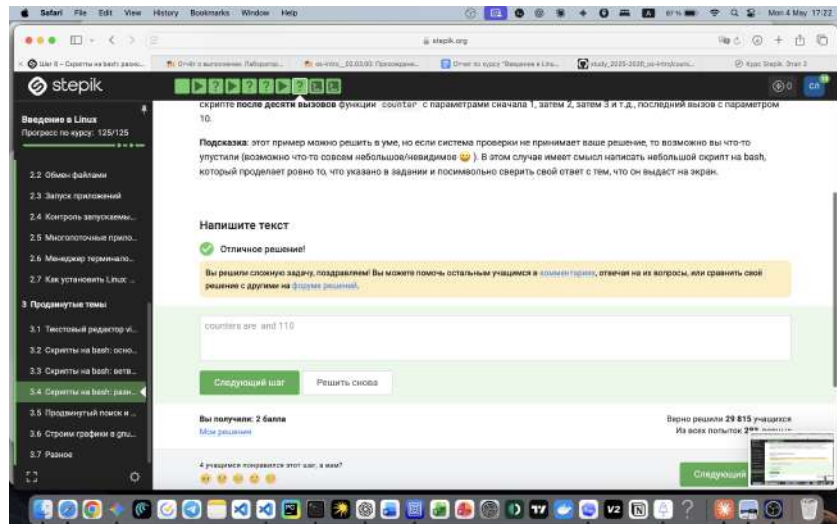


Рис. 3.21: Вопрос о выводе после вызовов функции

3.4.5 Шаг 22. Функция НОД по алгоритму Евклида

Практическое задание: написать функцию для вычисления наибольшего общего делителя по алгоритму Евклида. Реализована рекурсивная функция `gcd()` с основным циклом программы (рис. 3.22).

```
#!/bin/bash
```

```
gcd () {
    local m=$1
    local n=$2
    if [[ $m -eq $n ]]
    then
        echo "GCD is $m"
    elif [[ $m -gt $n ]]
    then
        gcd $((m - n)) $n
    else
        gcd $m $((n - m))
    fi
}
```

```

fi
}

while true
do
    read var1 var2
    if [[ -z "$var1" ]]
    then
        echo "bye"
        break
    fi
    gcd $var1 $var2
done

```

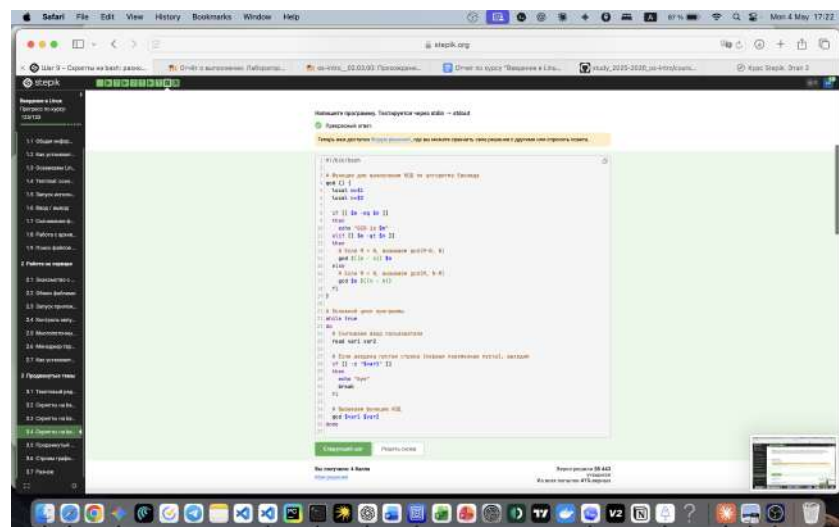


Рис. 3.22: Задание на функцию НОД

3.4.6 Шаг 23. Калькулятор для двух чисел

Практическое задание: написать калькулятор, принимающий два числа и оператор (3 аргумента) или команду exit. Скрипт реализован с чтением ввода, оператором case \$b in и вычислением через echo \$((a \$b c)) (рис. 3.23).

```
#!/bin/bash
```

```
while true
do
    read a b c d
    if [[ "$a" == "exit" && -z "$b" ]]; then
        echo "bye"
        break
    fi
    if [[ -n "$a" && -n "$b" && -n "$c" && -z "$d" ]]; then
        case $b in
            "+" | "-" | "*" | "/" | "%" | "**")
                echo $(( a $b c ))
                ;;
            *)
                echo "error"
                exit 0
                ;;
        esac
    else
        echo "error"
        exit 0
    fi
done
```

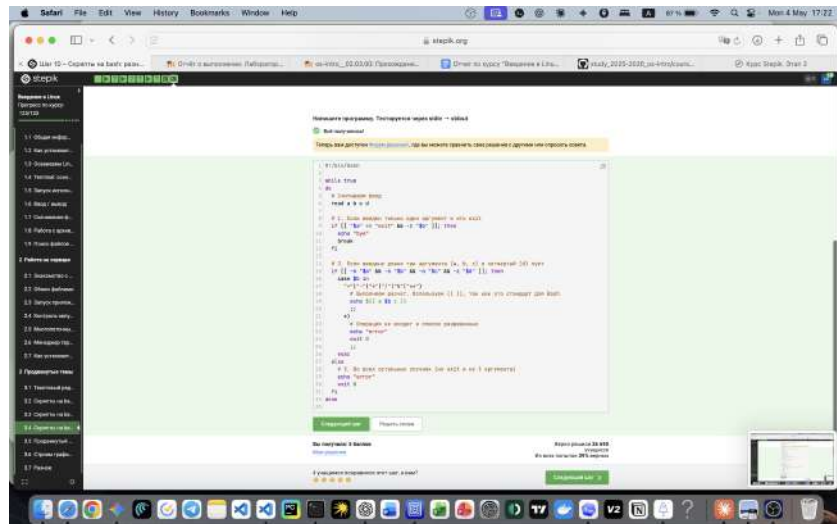



Рис. 3.23: Задание на калькулятор

3.5 Раздел 3.5. Продвинутый поиск и редактирование

3.5.1 Шаг 24. Опция -iname в команде find

Вопрос о том, какие файлы найдёт `find` с опцией `-iname "star"`, но не найдёт с `-name "star"`. Верные ответы: `STARS.txt` и `Star_Wars.avi` - опция `-iname` выполняет поиск без учёта регистра (рис. 3.24).

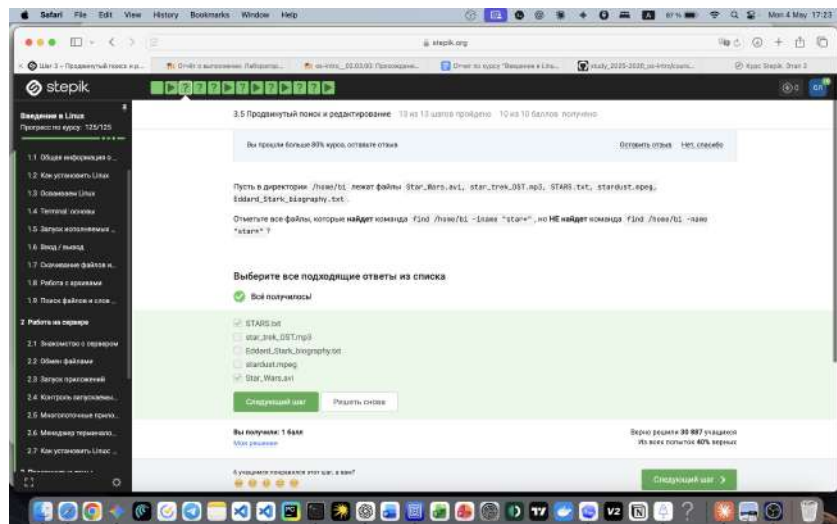


Рис. 3.24: Вопрос об опции -iname в find

3.5.2 Шаг 25. Опции -path и -name в команде find

Вопрос об утверждениях про опции -path и -name. Верные ответы: в некоторых случаях find с -name найдёт меньше файлов, чем с -path; если заменить -name на -path, результат иногда может остаться таким же (рис. 3.25).

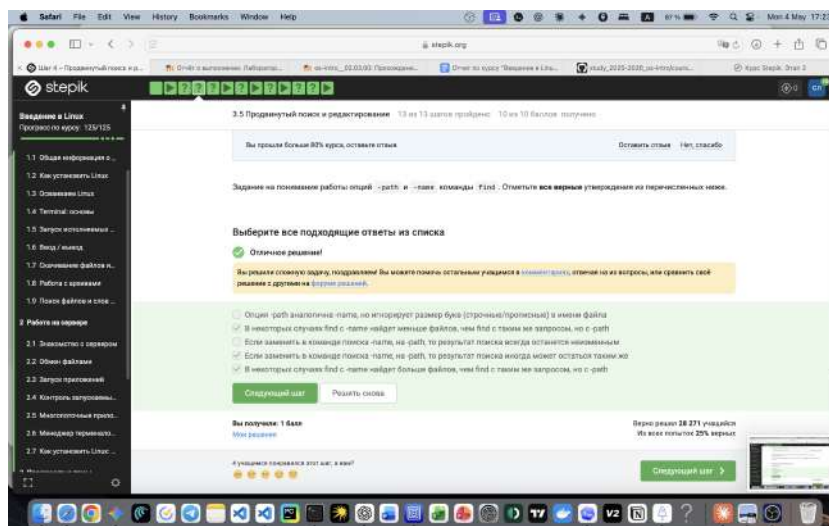


Рис. 3.25: Вопрос об опциях -path и -name в find

3.5.3 Шаг 26. Опции -mindepth и -maxdepth в find

Задание: определить, какие файлы будут найдены по команде `find ... -mindepth 2 -maxdepth 3 -name "file*"`. Верный ответ: **все кроме file3** - file3 не попадает в указанный диапазон глубины (рис. 3.26).

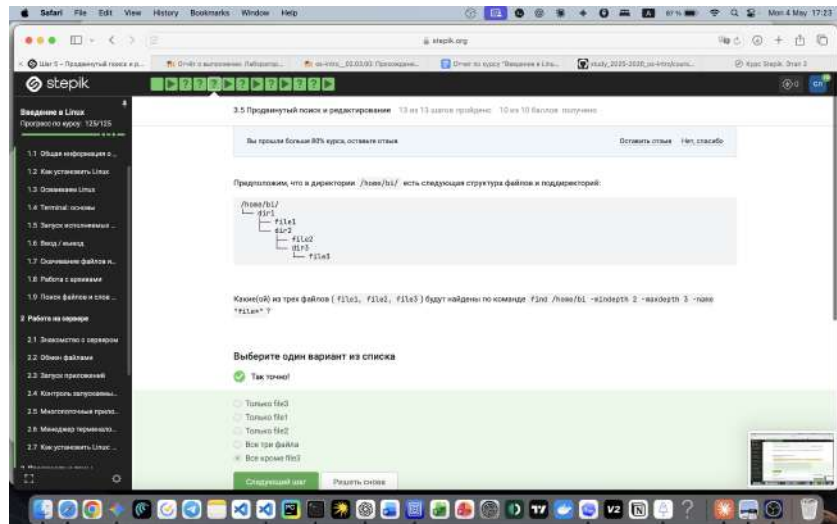


Рис. 3.26: Вопрос об опциях -mindepth и -maxdepth

3.5.4 Шаг 27. Опции -A, -B, -C в команде grep

Вопрос о том, какая команда грейп с опциями -A, -B, -C создаст файл наибольшего размера. Верный ответ: **results.txt будет одинакового размера во всех случаях** - при одинаковом количестве строк контекста итоговый размер одинаков (рис. 3.27).

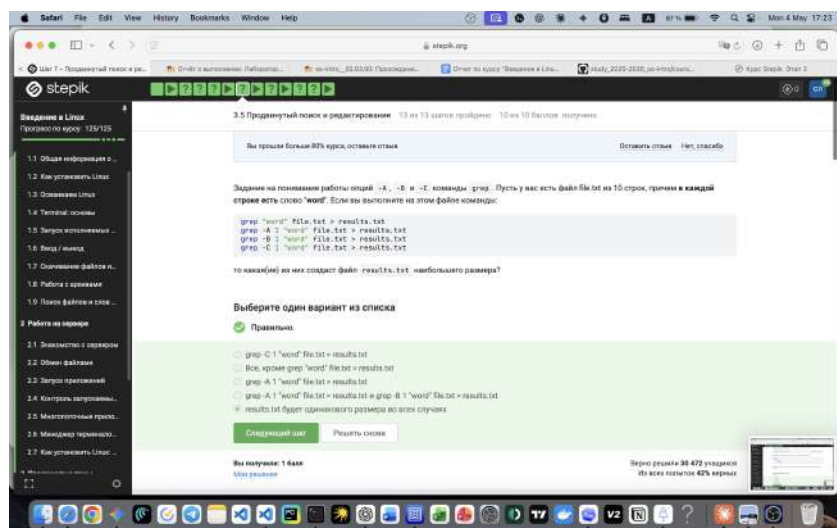


Рис. 3.27: Вопрос об опциях -A, -B, -C в grep

3.5.5 Шаг 28. Регулярное выражение в grep -E

Задание: определить, какие строки выведет команда `grep -E "[xkLXL]?[uU]buntu$" text.txt`. Верный ответ: **The best OS is Xubuntu** - строка соответствует шаблону (рис. 3.28).

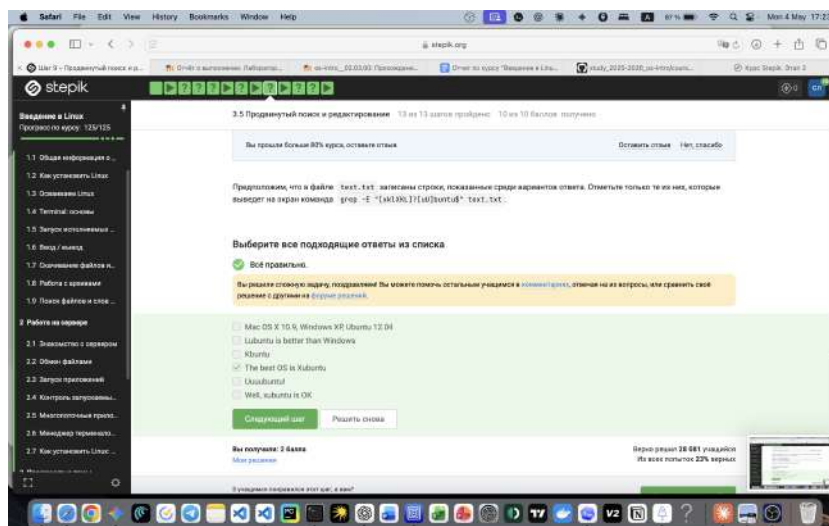


Рис. 3.28: Вопрос о регулярном выражении в grep -E

3.5.6 Шаг 29. Поведение sed без опции -n

Вопрос о том, что произойдёт, если в команде `sed -n "/[a-z]*р/"` убрать опцию `-n`. Верный ответ: **каждая строка будет выведена два раза** - без `-n` `sed` выводит все строки по умолчанию, а команда `р` добавляет ещё один вывод (рис. 3.29).

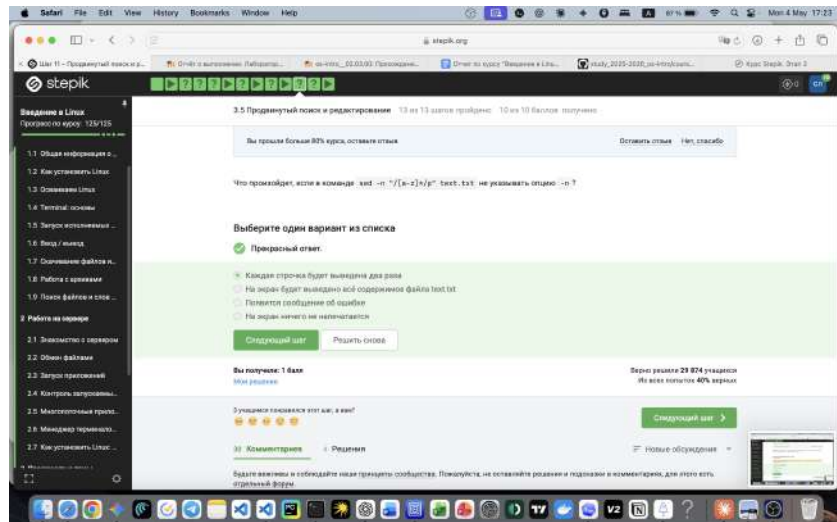


Рис. 3.29: Вопрос о поведении sed без опции -n

3.5.7 Шаг 30. Замена аббревиатур через sed

Задание: написать инструкцию sed, заменяющую аббревиатуры (последовательности заглавных букв) на слово abbreviation. Введён верный ответ:

```
sed 's/ [A-Z]\([A-Z]\)+ / abbreviation /g' input.txt > edited.txt
```

(рис. 3.30).

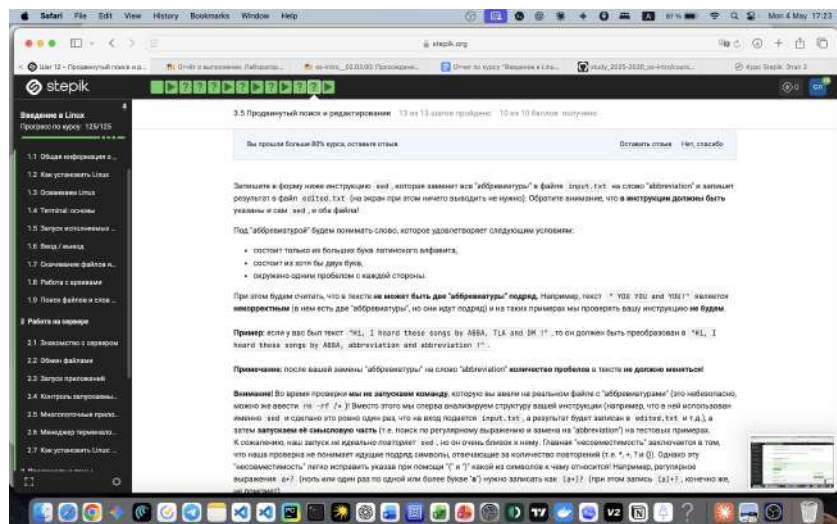


Рис. 3.30: Задание на замену аббревиатур через sed

3.5.8 Шаг 31. Продолжение задания по sed

Дополнение к заданию о замене аббревиатур - подтверждена корректность команды sed (рис. 3.31).

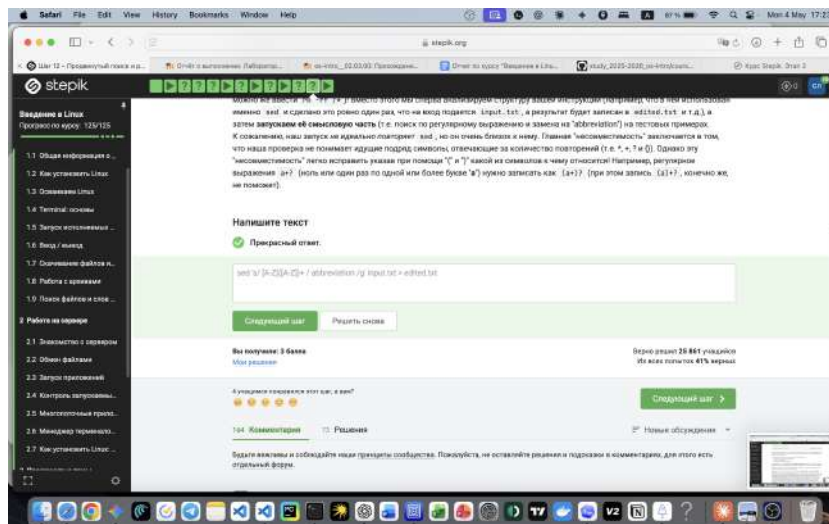


Рис. 3.31: Продолжение задания по sed

3.6 Раздел 3.6. Строим графики в gnuplot

3.6.1 Шаг 32. Опция gnuplot для сохранения графиков

Вопрос об опции gnuplot, предотвращающей автоматическое закрытие графиков при выходе из программы. Верный ответ: **-p** или **--persist** - эта опция оставляет окна с графиками открытыми (рис. 3.32).

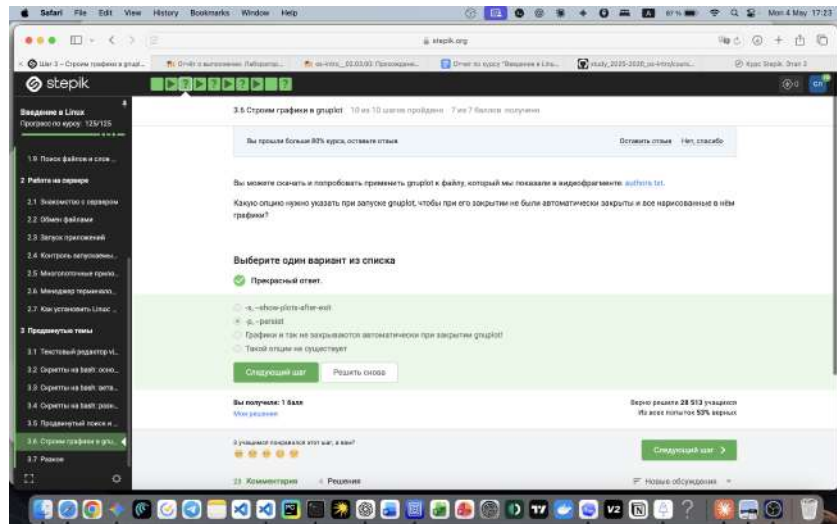


Рис. 3.32: Вопрос об опции `-persist` в `gnuplot`

3.6.2 Шаг 33. Команда `set key autotitle columnhead`

Задание: определить название ряда данных и количество точек при команде `set key autotitle columnhead`, если в файле нет заголовков. Верный ответ: название - первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена как заголовок) (рис. 3.33).

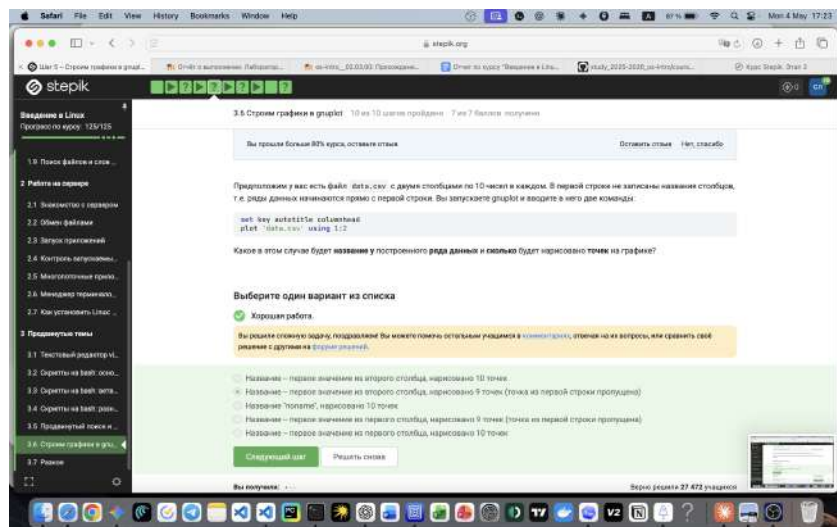


Рис. 3.33: Вопрос о команде `set key autotitle columnhead`

3.6.3 Шаг 34. Конкатенация строк для подписей на оси OX

Задание: составить команду gnuplot с конкатенацией для оформления подписей делений на оси OX. Введён верный ответ:

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

(рис. 3.34).

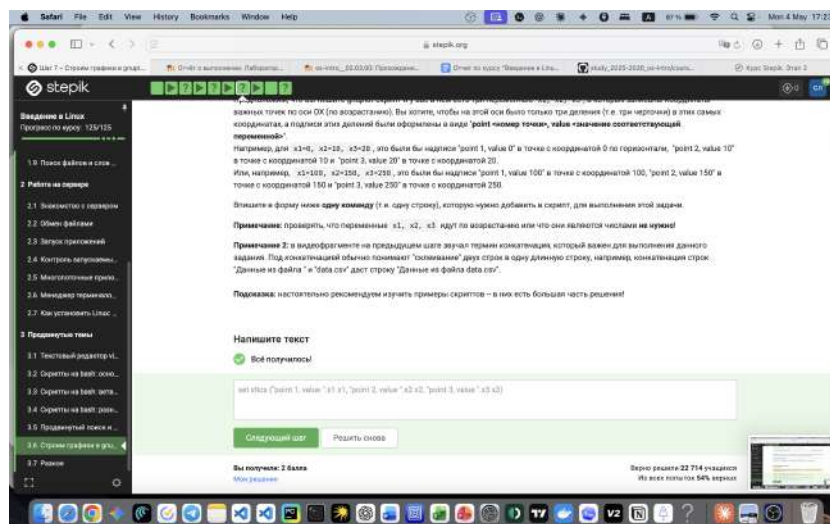


Рис. 3.34: Задание на конкатенацию строк для подписей осей

3.6.4 Шаг 35. Модификация скрипта move.rot в gnuplot

Задание: изменить скрипт move.rot так, чтобы график отражался зеркально, вращался в обратную сторону и в два раза быстрее. Введён верный ответ (рис. 3.35):

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

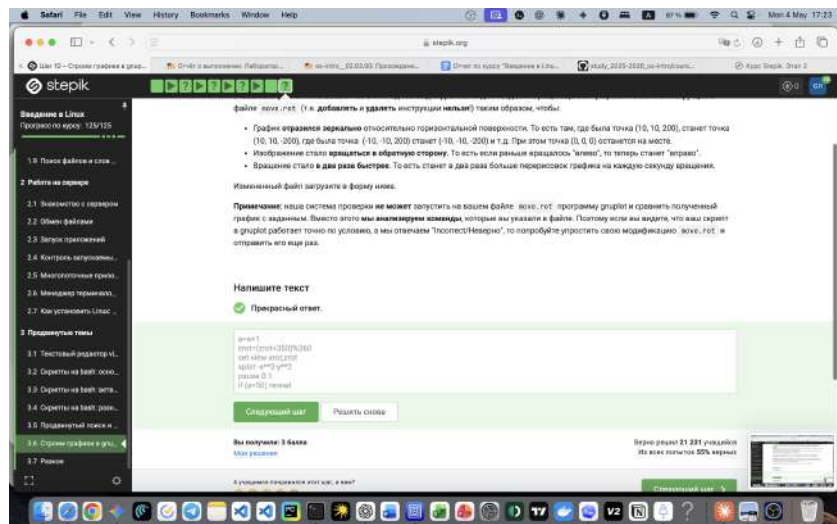



Рис. 3.35: Задание на модификацию скрипта move.rot

4 Выводы

В ходе прохождения третьей части курса на платформе Stepik освоены продвинутые темы работы в Linux. Изучен текстовый редактор vim: режимы работы, перемещение по словам (w, W), режим выделения Visual, команды замены через :%s. Написаны скрипты на bash: работа с переменными, позиционными аргументами, условиями if и case, циклами с continue, функциями и рекурсией. Реализованы практические программы: калькулятор и вычислитель НОД по алгоритму Евклида. Освоены продвинутые инструменты поиска и редактирования: find с опциями -iname, -mindepth, -maxdepth, grep -E с регулярными выражениями, sed с командами замены и опцией -n. Изучено построение графиков в gnuplot: опции запуска, подписи осей, анимация через скрипты. Все 35 шагов третьей части пройдены успешно.

Список литературы